Universidade do Algarve | Parallel and Distributed Systems | 2026

HW2: Social Networking in a Distributed Environment

Nome: _____ №:_____

Goal. The main goal of this lab is for students to get acquainted with the modelling of software architectures for distributed systems (the client-server architecture).

1 Before starting

(**a**) Open Rodin and import the recitation03.zip project that is available from Tutoria. Do not decompress the zip file but import it as an archive file.
(**b**) You will need to submit a Rodin project in the form of a zip file as your response to this lab. You will not need to submit a pdf file.
(**c**) The recitation03.zip Rodin project includes two machines, a poporo abstract machine with a simplified model of Facebook, and a poporo_distrib machine with a model for publishing content in a distributed environment. **You will not need, nor are you allowed, to edit either of these two machines**.

2 Poporo's abstract machine

The Rodin project includes two machines, a poporo abstract machine with a simplistic model of Facebook, and a poporo_distrib machine, which is the first refinement of poporo. The abstract machine only includes an operation (event), upload, which publishes the content item c in the page of the person p. The abstract machine does not know about the *mechanics* of how the content item c is published in a **distributed environment**, but the poporo_distrib machine does. The abstract machine knows the final effect (result) of publishing a content item, for instance, the content := content ∪ {c} action in the upload event of the abstract machine indicates that eventually content c is added to the set of published content items. The abstract machine does not say the sequence of steps to reach the state in which c is included in the set of content items. The poporo_distrib concrete machine tells us about the sequence of steps to reach that state. In a first stage, the event send_upload makes a request for the content c to be published in the page of the person p, and, in a second stage, the event process_upload processes the request so that the content is published in the page of the person p.

The machine poporo_distrib is shown in Figure 1. The variable pages and event upload are defined the same as in the previous homework assignment. The event upload publishes the content item c in the page of the person p. A content item may be a picture, text, or video that p wants to publish in their page.

3 Poporo in distributed environment

Machine poporo_distrib models Poporo (our model for Facebook) in a general, unconstrained version of a distributed environment. In Figure 2, variable content_c keeps the content items that have been first requested to be published then published. Event send_upload requests a content item c to be published and event process_upload (if executed successfully) publishes the content. The event send_upload modifies content (since it extends upload) but it does not modify content_c. The event process_upload modifies content_c but it does not modify content. Therefore, after send_upload is executed, variable content has elements that are not yet in content_c.

The variable pages_c (see invariant @invr1_1) is the concrete version of the variable pages: pages_c is to pages what content_c is to content. Variable pub_req keeps track of the requests for content items to be

```
machine poporo sees ctx1
variables person // persons in the network
    content // content items in the network
    pages // page content per person
invariants
  @inv1 person ⊆ PERSON
  @inv2 content ⊆ CONTENT
  @inv3 pages ∈ content ⇸ person

events
  event INITIALISATION
    then
      @act1 person ≔ ∅
      @act2 content ≔ ∅
      @act3 pages ≔  ∅
  end

  event upload
    any c p
    where
      @grd1 p ∈ person
      @grd3 c ∈ CONTENT\content
    then
      @act1 content ≔ content ∪ {c}
      @act2 pages ≔ pages ∪ {c↦p}
  end
end
```

Figure 1: Rodin IDE

```
machine poporo_distrib refines poporo  sees ctx1

variables person // persons in the network
          content // content items in the net
          pages // page content per person
          pub_req // publishing request
          content_c // concrete version of set content
          pages_c // concrete version of pages

invariants
  @invr1_0 content_c ⊆ CONTENT // concrete version of content
  @invr1_1 pages_c ∈ content_c ↔ person // concrete version of pages
  @invr1_2 pub_req ∈ CONTENT ⇸ person // pending content per person who publishes it
  @invr1_3 dom(pub_req) ∩ content_c = ∅ // pending content items are not yet published
  // two gluing invariants
  @gluer11 content = content_c ∪ dom(pub_req) // it relates content_c to content
  @gluer12 pages = pages_c ∪ pub_req // it relates pages_c to pages
```

Figure 2: Rodin IDE

published and the server to which the request is sent (see invariant @invr1_2). Invariant @invr1_3 states that content items in content_c have not yet been published.

There is a fundamental concept in Event-B and that is of a **gluing invariant**. A gluing invariant relates the state of the concrete machine to the state of the abstract machine so that the concrete machine is not a independent of or unrelated to the abstract machine. The concrete machine needs to behave according to the abstract machine's behaviour. The concrete machine poporo_distrib declares two gluing invariants. Invariant @gluer11 relates content_c to content. Invariant @gluer12 relates pages_c to pages. Invariant @gluer11 says that the content items that have not yet been published are in dom(pub_req). Invariant @gluer12 says that pages are either up-to-date or the last changes are kept in pub_req.

Figure 3 shows the two events of machine poporo_distrib. Event send_upload places a request for content c to be published in the page of person p. Guard @grdr1_1 expresses that the content item c must be fresh. Event send_upload implicitly accounts for the guards and actions of event upload of the abstract machine as send_upload extends upload. Event process_upload processes an upload request. Guard @grd1 makes sure that c is unpublished content. Guard @grd2 states that p made the request to publish c in their page. Event action @act1 removes the request, and action @act3 handles the request.

4 This homework assignment

Notice that there is no assumption on the way requests are handled, in particular, nothing is said about the architecture of Poporo. Requests could well be handled by a server that decides on whether or not the content is published on the network. Also notice that, for instance, content represents the final state of network content in a distributed environment whereas content_c represents an intermediate state in which the content has not yet been published. The same goes for pages_c and pages.

In this homework assignment, **you will create an Event-B model for publishing content in a distributed environment in which we assume a client-server architecture**. You will not edit not edit the machines poporo or poporo_distrib. The machine poporo_distrib will serve as reference to write your model. Make sure all the proof obligations are discharged. The number of undischarged proof-obligations is a good measure of your grade as typically a simple and sound model shall have none. Make sure you have installed the SMT plugin (see previous homework assignment) so that you can discharge all your proof obligations. **You are not allowed to use quantified expressions either universally or existentially**.

(**a**) Open Rodin version 3.5 and import the file recitation03.zip as an existing project into your workspace. Make sure you do not import the project as a folder but as an archive file.

```
event send_upload extends upload
  where
    @grdr1_1 c ∉ dom(pub_req)
  then
    @actr1_1 pub_req ≔ pub_req ∪ {c ↦ p}
end

event process_upload
  any c p
  where
    @grd1 p∈person ∧ c∈dom(pub_req)
    @grd2 pub_req(c) = p
  then
    @act1 pub_req ≔ pub_req \ {c ↦ p}
    @act2 content_c ≔ content_c ∪ {c}
    @act3 pages_c ≔ pages_c ∪ {c ↦ p}
end
```

Figure 3: Rodin IDE

(**b**) The next step is to create a new context that extends ctx1. Right-click ctx1, select *Extend*, and name your context ctx2. Context ctx2 will declare the carrier set SERVER that will help us type the set of registered servers.

(**c**) The next step is to create a machine poporo_clientserver as a refinement of the poporo machine. The machine poporo_clientserver will contain your encoding for a distributed client-server model of Poporo. Right-click poporo, select *Refine*, and name the machine poporo_clientserver.

(**d**) Add the following variables to the poporo_clientserver machine: (*i*) server is a set that represents the set of registered servers, (*ii*) content_c is the concrete version of content, (*iii*) pages_c is the concrete version of pages, (*iv*) pub_req is similar to pub_req in poporo_distrib except that it must further account for information on the server to whom the request is sent, (*v*) variable pub_ack keeps track of the requests that have been acknowledged by a server, (*vi*) variable pending_content keeps track of the content that is pending acknowledgement, and (*vii*) variable pending_pages keeps track of the pages that have not yet been updated.

(**e**) Add invariants that relate the machine variables defined in the previous step. You absolutely need to include two **gluing invariants**, one that relates the concrete version of content with its abstract counterpart, and one that relates the concrete version of pages with its abstract counterpart.

(**f**) Add events for requesting that content be published and for a server accepting the request.

5 What to turn in

You should turn in a zip file with the Rodin project of your solution of this lab. First rename your recitation03 project as recitation03_a666666 (using your student number instead of 666666). Next, in Rodin, select your project, then `File`, `Export`, `General`, `Archive File`, `Next`, `Browse`, and save your file as recitation03_a666666.zip.

6 Grading

(**i**) Your Rodin project accounts for **20**% of the total grade, and the remaining **80%** corresponds to the oral defence of your homework assignment. The defence will assess your understanding of your model, your design decisions, and your ability to explain and justify your solution.

(**ii**) The grade of the Rodin project (**20**%) will be based on the following criteria:

- **Completeness.** The model includes all required components of the client-server architecture and all required functionality.
- **Soundness.** All proof obligations are discharged.
- **Refinement correctness.** The concrete machine correctly refines the abstract machine, and the gluing invariants properly relate abstract and concrete variables.
- **Consistency.** The model represents a client-server architecture (servers, requests, acknowledgements) and does not model a different architecture.
- **Modelling quality and clarity.** The model is well-structured, readable, and uses meaningful variables, invariants, events without unnecessary complexity.

(**iii**) **The oral defence (80%) will evaluate your understanding of the model. You must be able to explain the purpose of each variable, invariant, gluing invariant, and event, justify your modelling decisions, and explain why your model is correct. Defences will take place the week of March 9 during the respective PLs in which students are enrolled.**.